

SLIDE 1

YOUR DATABASE PASSWORD IS **IN** **YOUR CODE!**



DEVELOPER



GITHUB
REPOSITORY



```
application.yml
spring:
  datasource:
    url: jdbc:mysql://mydb
    username: admin
    password: Admin@123
```

**EXPOSED
SECRET!**



WHAT IF THIS REPOSITORY
LEAKS?



AWS SECRETS MANAGER



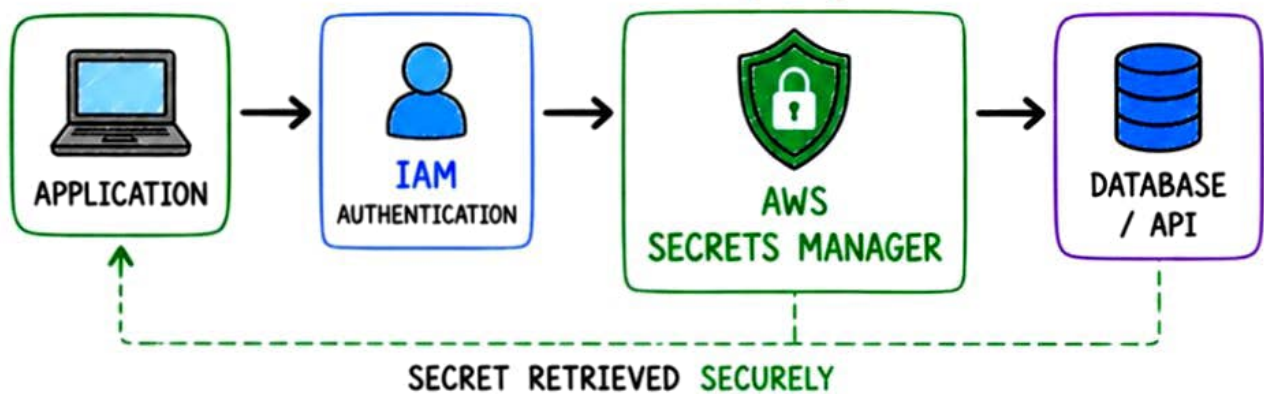
Secrets belong in a secret store –
NOT in source code.

SLIDE 2



WHAT IS AWS SECRETS MANAGER?

A managed AWS service for securely **storing**, **retrieving** and **rotating** secrets.



EXAMPLES OF SECRETS

- Database credentials (user & password)
- API keys
- Application credentials
- OAuth tokens
- Other sensitive configuration

KEY BENEFITS

- ✓ Centralized & secure storage
- ✓ Fine-grained access with IAM
- ✓ Automatic secret rotation
- ✓ Encryption with AWS KMS
- ✓ Audit & monitoring with CloudTrail



APPLICATION RETRIEVES THE SECRET
AT RUNTIME – **NOT AT BUILD TIME.**

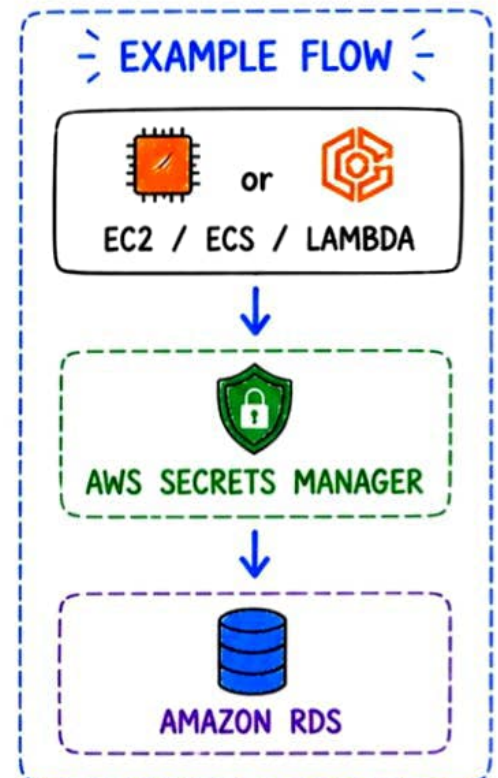
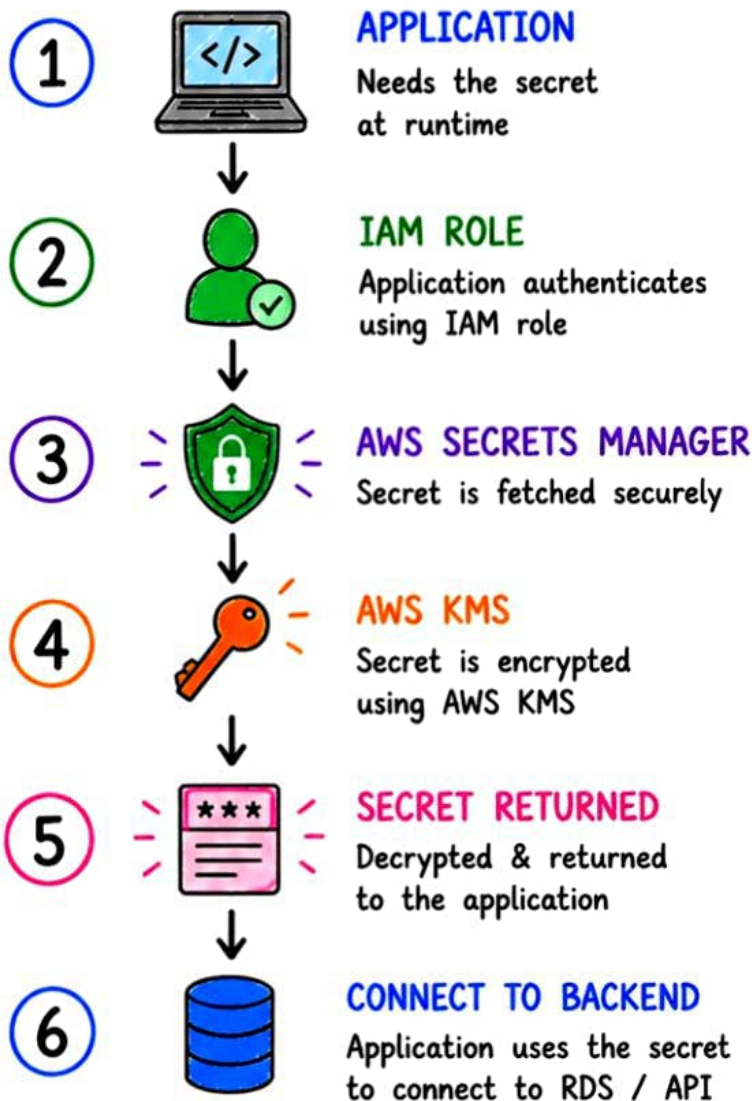


SLIDE 3



HOW DOES IT WORK?

YOUR APPLICATION GETS A SECRET



KEY IDEA

- ✓ No hardcoded credentials
- ✓ No secrets in code
- ✓ Retrieve securely at runtime



SECURE, SIMPLE & AUTOMATED WAY
TO **MANAGE** YOUR **SECRETS**.

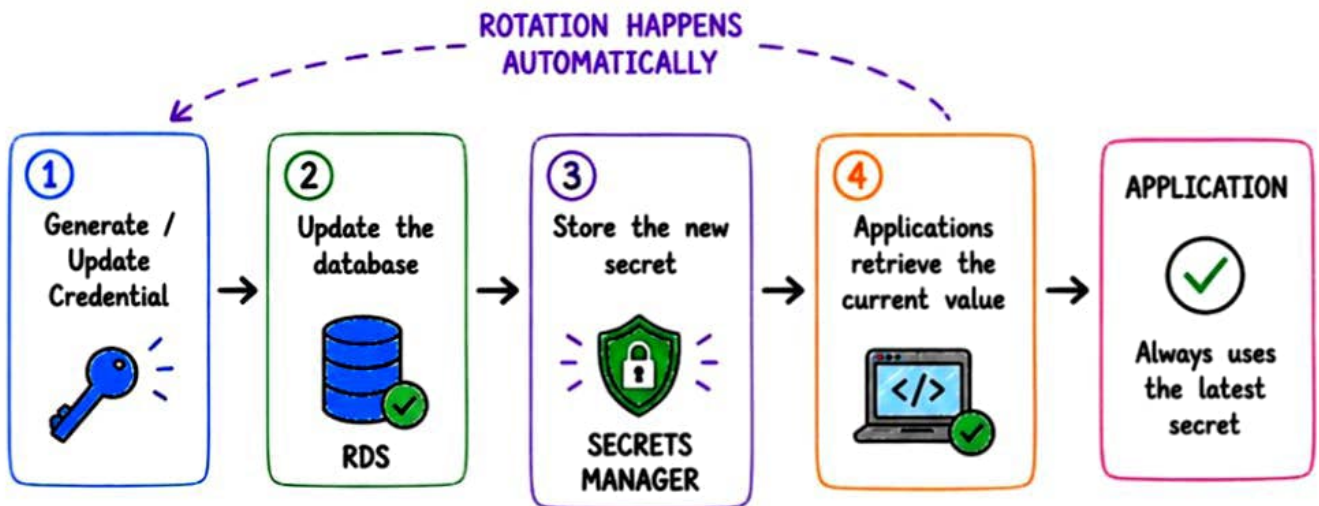


SLIDE 4



AUTOMATIC SECRET ROTATION

Rotate secrets automatically & keep your application running **without interruption!**



Secrets Manager rotation Lambda function updates the secret in the target service (like RDS) and keeps your application running with **zero downtime**.

BENEFITS



Automatically rotate secrets



Improve security posture



No manual intervention



Reduce risk of credential leaks



ROTATE OFTEN. **STAY SECURE.** STAY AHEAD.



SLIDE 5



SECRETS MANAGER VS PARAMETER STORE

WHICH ONE SHOULD YOU USE?

FEATURE / CAPABILITY	 AWS SECRETS MANAGER	 AWS PARAMETER STORE
 Primary purpose	Store, manage & rotate secrets	Store configuration & parameters
 Store secrets	✓ Yes	✓ Yes (SecureString)
 Automatic rotation	✓ Built-in rotation	✗ Manual / DIY
 Database credential rotation	✓ Native support (RDS, etc.)	✗ DIY with Lambda
 Cost	Paid service	Standard tier is low-cost
 Best for	Secrets lifecycle, rotation & auditing	App configs, feature flags, non-rotating parameters
 Scaling & operations	Managed, highly scalable, minimal ops	Highly scalable, serverless
 Integration with AWS services	Deep integration with RDS, Redshift, etc.	Works across many AWS services

USE CASE TIP



Use **SECRETS MANAGER** when you need secret management + rotation.



Example: RDS password, API keys, DB credentials



Use **PARAMETER STORE** when you need config management or simple parameters.



Example: App settings, feature flags, endpoints



Right tool. Right use case. Better **Security**. Lower **Cost**.

THAT'S CLOUD SMARTNESS!



SLIDE 6



INTERVIEW CHEAT SHEET



HOW WOULD YOU STORE AN RDS PASSWORD IN PRODUCTION?

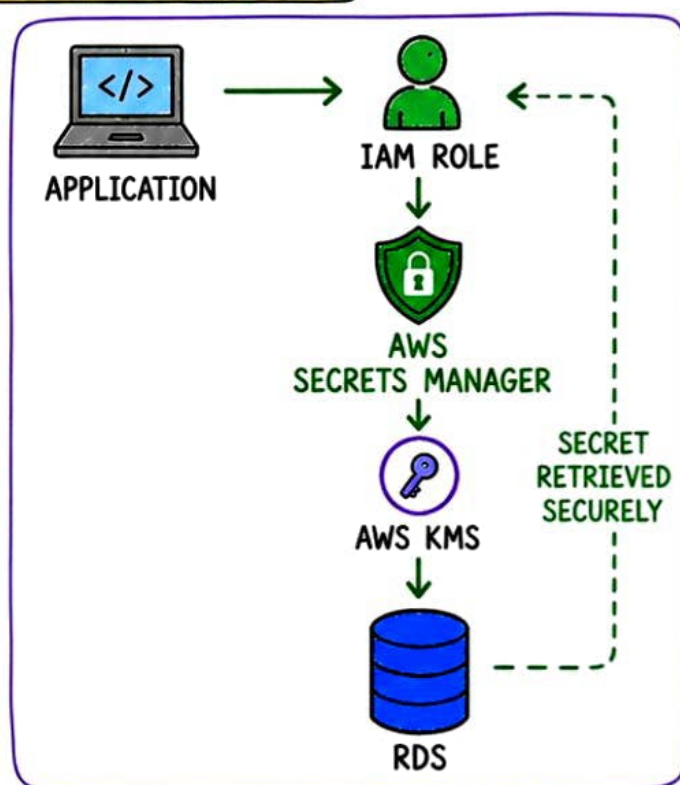


STRONG ANSWER FRAMEWORK

- 1 Store the password in **AWS SECRETS MANAGER**.
- 2 Encrypt the secret using **AWS KMS**.
- 3 Grant access using **IAM role** (least privilege).
- 4 Application retrieves secret at **runtime**.
- 5 Enable automatic **rotation** to keep it secure.

WHY THIS IS THE BEST WAY?

- ✓ No hardcoded credentials
- ✓ No secrets in Git or image
- ✓ Centralized, secure & auditable
- ✓ Automatic rotation support
- ✓ Highly available & scalable



COMMON MISTAKES TO AVOID



Hardcoding secrets in code



Storing secrets in Git



Over-permissive IAM policies



Not rotating secrets



SECURE YOUR SECRETS.
PROTECT YOUR APPLICATION.
BUILD WITH CONFIDENCE.



SAVE THIS SLIDE

SHARE IT



FOLLOW FOR MORE CLOUD KNOWLEDGE!

